

Tentu, mari kita lanjut ke file berikutnya: **Robert.cs**.

Jika **Canny** adalah yang paling cerdas dan **Prewitt** adalah yang klasik standar, maka **Robert (Robert Cross)** adalah "sesepuh" atau algoritma deteksi tepi yang paling tua (1963) dan paling sederhana.

Berikut adalah bedah kodenya untuk persiapan UAS Anda.

1. Gambaran Besar: Apa itu Robert Cross?

Robert berbeda dari Prewitt atau Sobel karena dia **tidak menggunakan matriks 3x3**, melainkan **2x2**.

- Dia melihat perbedaan warna secara **diagonal** (silang).
- Karena hanya melihat area yang sangat kecil (2x2), dia **sangat cepat** tapi **sangat sensitif terhadap noise** (bintik debu sedikit saja dianggap garis).

2. Penjelasan Kode Per Bagian

A. Persiapan Memori (LockBits)

Bagian ini standar di semua file pemrosesan gambar performa tinggi.

C#

```
public Bitmap Apply(Bitmap source)
{
    // Clone source agar aman
    Bitmap bmp = new Bitmap(source);
    // ... (Setup LockBits standar) ...

    // Siapkan array byte untuk membaca data pixel
    Marshal.Copy(srcData.Scan0, pixelBuffer, 0, bytes);
}
```

Poin UAS:

- Sama seperti sebelumnya, kita mengunci memori (LockBits) karena kita akan melakukan jutaan operasi matematika. Akses langsung ke array byte jauh lebih cepat daripada fungsi bawaan C#.
-

B. Loop Utama (Perbedaan Penting)

Perhatikan batas loop-nya.

C#

```
// Batas loop dikurangi 1 (width - 1)
// Kenapa? Karena Robert mengintip pixel "di depannya" (x+1).
// Kalau kita loop sampai ujung, nanti error "Index Out Of Range".
Parallel.For(0, height - 1, y =>
{
    for (int x = 0; x < width - 1; x++)
    {
        // ...
    }
});
```

C. Inti Algoritma: Operasi Silang 2x2

Inilah ciri khas Robert. Dia mengambil 4 pixel yang membentuk kotak kecil.

C#

```
// Hitung posisi pixel di array 1D
int k = y * stride + x * 4; // P1: Kiri-Atas
int k_right = y * stride + (x + 1) * 4; // P2: Kanan-Atas
int k_down = (y + 1) * stride + x * 4; // P3: Kiri-Bawah
int k_diag = (y + 1) * stride + (x + 1) * 4; // P4: Kanan-Bawah (Diagonal)

// Ambil nilai intensitasnya (Blue channel)
int p1 = pixelBuffer[k];
int p2 = pixelBuffer[k_right];
int p3 = pixelBuffer[k_down];
int p4 = pixelBuffer[k_diag];
```

Visualisasinya seperti ini:

Plaintext

```
[ P1 ] [ P2 ]  
[ P3 ] [ P4 ]
```

D. Rumus Matematika (Robert Cross)

Robert menghitung selisih secara silang.

C#

```
// Gx: Selisih Diagonal Utama (Kiri Atas vs Kanan Bawah)
```

```
int gx = Math.Abs(p1 - p4);
```

```
// Gy: Selisih Diagonal Sekunder (Kanan Atas vs Kiri Bawah)
```

```
int gy = Math.Abs(p2 - p3);
```

```
// Magnitude (Kekuatan Garis)
```

```
// Di sini menggunakan pendekatan "Manhattan Distance" (Penjumlahan biasa)
```

```
// Rumus aslinya biasanya Euclidean: sqrt(gx^2 + gy^2)
```

```
// Tapi penjumlahan biasa (gx + gy) jauh lebih cepat dihitung komputer.
```

```
int magnitude = gx + gy;
```

E. Clamping & Penyimpanan

Seperti biasa, pastikan nilainya tidak "jebol" batas warna (0-255).

C#

```
if (magnitude > 255) magnitude = 255;
if (magnitude < 0) magnitude = 0;

// Simpan hasilnya
resultBuffer[k] = (byte)magnitude; // B
resultBuffer[k + 1] = (byte)magnitude; // G
resultBuffer[k + 2] = (byte)magnitude; // R
resultBuffer[k + 3] = 255; // Alpha
```

Simulasi Pertanyaan Sidang/UAS untuk Robert.cs

1. Pertanyaan Konsep (Paling Sering)

Dosen: "Kenapa Robert menggunakan matriks 2x2 sedangkan yang lain (Sobel/Prewitt) pakai 3x3? Apa kelebihan dan kekurangannya?"

Jawaban Anda:

"Robert menggunakan 2x2 karena tujuannya adalah **kecepatan maksimal**. Dia hanya butuh 4 pixel untuk satu perhitungan.

- **Kelebihan:** Sangat cepat dan efisien.
- **Kekurangan:** Sangat sensitif terhadap *noise* (bintik kotor). Karena area pandangnya sempit, bintik kecil pun akan dianggap sebagai garis tajam. Beda dengan 3x3 yang bisa 'merata-ratakan' noise di sekitarnya."

2. Pertanyaan Matematika

Dosen: "Di kode ini tertulis $\text{magnitude} = g_x + g_y$. Bukankah rumus vektor harusnya pakai Akar Kuadrat (Math.Sqrt)?"

Jawaban Anda:

"Betul, Pak. Secara teori matematis yang presisi harusnya menggunakan Euclidean Distance ($\text{Sqrt}(x^2 + y^2)$). Namun, di kode ini kita menggunakan Manhattan Distance ($|x| + |y|$) sebagai pendekatan (aproksimasi). Tujuannya adalah performa. Operasi akar kuadrat itu berat bagi CPU. Dengan penjumlahan biasa, hasilnya mirip tapi komputasinya jauh lebih ringan."

3. Pertanyaan Teknis: Indexing

Dosen: "Kenapa di Robert variabelnya banyak sekali? Ada k , k_right , k_down , k_diag ? Di Prewitt tidak seribet ini?"

Jawaban Anda:

"Karena Robert mengakses pixel secara 'manual' berdasarkan posisi relatif tetangganya dalam format Array 1 Dimensi.

- k adalah titik saat ini (x, y) .
- k_right adalah $(x+1, y)$ atau geser 4 byte ke kanan.
- k_down adalah $(x, y+1)$ atau geser satu baris (stride) ke bawah.

Di Prewitt kita menggunakan loop bersarang ky dan kx yang lebih rapi, tapi untuk Robert yang cuma 4 titik, menulis manual seperti ini justru lebih efisien (hardcoded)."

4. Pertanyaan Deteksi Arah

Dosen: *"Apakah Robert bisa mendeteksi garis vertikal tegak lurus dengan baik?"*

Jawaban Anda:

"Kurang baik, Pak.

Karena rumus Robert itu $|P1 - P4|$ (Diagonal), dia paling responsif terhadap garis yang miring 45 derajat.

Kalau garisnya tegak lurus vertikal atau horizontal murni, respon Robert sedikit lebih lemah dibandingkan Prewitt atau Sobel yang memang punya kernel khusus vertikal/horizontal."

Apakah Anda sudah merasa cukup dengan penjelasan file-file deteksi tepi ini, atau kita mau kembali membahas **File Lain** yang mungkin belum terbahas (seperti Helper atau Logic lainnya)?